

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE
SÃO PAULO**

VINÍCIUS DE MORAIS ALMEIDA SANTOS

**SISTEMA DE GERENCIAMENTO DE BANCO DE DADOS
PARA UM E-COMMERCE DE VESTUÁRIO**

CAMPOS DO JORDÃO

2026

RESUMO

A gestão eficiente de dados em um e-commerce é um processo crítico para garantir a integridade das operações comerciais, o controle de estoque e o acompanhamento financeiro. O presente trabalho tem como objetivo projetar, desenvolver e implementar um banco de dados relacional robusto para o gerenciamento centralizado de uma loja virtual de vestuário. A metodologia adotada abrangeu desde o levantamento de requisitos e definição de regras de negócio, passando pela modelagem lógica e conceitual no padrão DBML (ferramenta dbdiagram.io), até a implementação física utilizando a linguagem T-SQL no Microsoft SQL Server. O sistema desenvolvido contempla 6 tabelas normalizadas (Cliente, Categoria, Produto, Pedido, Item_Pedido e Pagamento), garantindo a eliminação de redundâncias e assegurando a consistência transacional por meio de chaves primárias e estrangeiras. Para validar a eficiência da arquitetura, uma carga massiva e realista de dados foi injetada, compreendendo 100 clientes, 30 produtos estratégicos e 1.000 pedidos gerados via laço de repetição. Como resultado, elaborou-se um conjunto de 30 consultas analíticas complexas, utilizando Common Table Expressions (CTEs), Window Functions e agregações, demonstrando a capacidade do banco de dados em extrair inteligência de negócios (Business Intelligence) para suporte à tomada de decisão.

Palavras-Chave: Banco de Dados Relacional; E-commerce; SQL Server; Modelagem de Dados; Business Intelligence.

ABSTRACT

Efficient data management in e-commerce is a critical process to ensure the integrity of commercial operations, inventory control, and financial tracking. The present work aims to design, develop, and implement a robust relational database for the centralized management of an online clothing store. The adopted methodology ranged from requirements gathering and business rules definition, through logical and conceptual modeling in the DBML standard (dbdiagram.io tool), to physical implementation using the T-SQL language in Microsoft SQL Server. The developed system comprises six normalized tables (Client, Category, Product, Order, Order_Item, and Payment), ensuring the elimination of redundancies and securing transactional consistency through primary and foreign keys. To validate the architecture's efficiency, a massive and realistic data load was injected, comprising 100 clients, 30 strategic products, and 1,000 orders generated via a repetition loop. As a result, a set of 30 complex analytical queries was developed, utilizing Common Table Expressions (CTEs), Window Functions, and aggregations, demonstrating the database's capability to extract Business Intelligence to support decision-making.

Keywords: Relational Database; E-commerce; SQL Server; Data Modeling; Business Intelligence.

LISTA DE ILUSTRAÇÕES

FIGURA 1 – Modelo Lógico do Banco de Dados

16

SUMÁRIO

1	INTRODUÇÃO	6
1.1	Objetivos	7
1.2	Justificativa	7
1.3	Aspectos Metodológicos	8
1.4	Aporte Teórico	9
2	FUNDAMENTAÇÃO TEÓRICA	10
2.1	O Modelo Relacional e as Restrições de Integridade	10
2.2	Normalização de Dados	10
3	PROJETO PROPOSTO	11
3.1	Considerações Iniciais	11
3.2	Requisitos e Regras de Negócio	11
3.3	Modelagem Lógica e Estrutura de Dados	17
3.4	Implementação Física (DDL)	17
3.5	Mecanismo de Carga e Injeção Massiva (DML)	32
4	AVALIAÇÃO	32
4.1	Consultas Analíticas (SQL)	32
5	CONCLUSÃO	40
	REFERÊNCIAS	42

1 INTRODUÇÃO

O comércio eletrônico (e-commerce) representa um dos setores de maior crescimento na economia global, exigindo infraestruturas tecnológicas capazes de processar altos volumes de transações com precisão e rapidez. Em um cenário de varejo online, a eficiência no controle de estoque, no rastreamento de pedidos e na reconciliação de pagamentos é fundamental para a saúde operacional e estratégica da empresa.

Este projeto tem por objetivo projetar e desenvolver um banco de dados relacional para o gerenciamento integrado de um e-commerce focado no nicho de vestuário e acessórios. A base de dados visa substituir controles informais, garantindo a integridade, segurança e consistência das informações transacionais, estabelecendo uma fonte única de verdade corporativa. Para a consecução deste objetivo, foram estabelecidos os seguintes objetivos específicos: levantar os requisitos e regras de negócio para uma loja de roupas; modelar a estrutura lógica e conceitual do banco de dados com no mínimo 5 tabelas; implementar a estrutura física no SGBD Microsoft SQL Server; popular as tabelas com uma carga de dados massiva e realista; e elaborar 30 consultas analíticas complexas.

A justificativa para o desenvolvimento recai sobre a necessidade técnica de dominar ferramentas avançadas de banco de dados (BDD2). A implementação não apenas atende a requisitos acadêmicos, mas simula um ambiente de produção real, capacitando a mitigação de riscos operacionais, o controle de concorrência e a extração de métricas de Business Intelligence (BI).

Metodologicamente, o trabalho caracteriza-se como uma pesquisa aplicada.

O desenvolvimento combinou fundamentos do Modelo Relacional com as fases clássicas de engenharia de dados: modelagem lógica e física, normalização (até a Terceira Forma Normal) e estruturação de *scripts* em Linguagem de Consulta Estruturada (SQL).

1.1 Objetivos

Este trabalho tem por objetivo geral projetar, desenvolver e implementar um banco de dados relacional robusto para o gerenciamento centralizado e integrado das operações de um e-commerce focado no segmento de vestuário e acessórios.

Para a consecução deste objetivo foram estabelecidos os seguintes objetivos específicos:

- Levantar os requisitos e as regras de negócio necessários para o mapeamento do fluxo operacional de uma loja virtual, contemplando o cadastro de clientes, o controle rigoroso de estoque e o rastreamento de transações financeiras;
- Modelar a estrutura conceitual e lógica do banco de dados, resultando em um esquema relacional normalizado até a Terceira Forma Normal (3FN), garantindo a eliminação de redundâncias;
- Implementar o modelo físico no Sistema de Gerenciamento de Banco de Dados (SGBD) Microsoft SQL Server, utilizando a linguagem T-SQL para a criação das tabelas e restrições de integridade (DDL);
- Desenvolver rotinas avançadas de inserção de dados (DML) para popular o sistema com uma carga massiva e orgânica, simulando cenários transacionais reais com a injeção de 1.000 pedidos;
- Elaborar e validar 30 consultas analíticas complexas, empregando recursos avançados da linguagem SQL (como Common Table Expressions, Window Functions e junções múltiplas), visando a extração de métricas de Business Intelligence (BI) para o suporte à tomada de decisão.

1.2 Justificativa

A gestão de operações no comércio eletrônico de vestuário e acessórios impõe desafios complexos devido à necessidade de gerenciar múltiplos atributos por produto, como variações de tamanho e cor, além de um fluxo contínuo de pedidos e liquidações financeiras. A dependência de controles descentralizados ou informais insere riscos severos de redundância de dados, falhas de sincronia de estoque e inconsistências na reconciliação de pagamentos. Desse modo, o desenvolvimento deste projeto justifica-se pela centralização operacional em um modelo relacional

robusto, estruturado no Microsoft SQL Server, que assegura o cumprimento das propriedades ACID e estabelece uma fonte única da verdade para a organização. Sob a perspectiva acadêmica e prática da engenharia de dados, o projeto justifica-se pelo desafio de interagir com uma base de dados em escala corporativa. A simulação de um cenário transacional com mil pedidos e milhares de itens inseridos provê o ambiente ideal para o desenvolvimento de consultas analíticas complexas. A aplicação de recursos avançados da linguagem SQL, como Common Table Expressions (CTEs) e Window Functions, demonstra como a modelagem física estrita habilita a extração de métricas gerenciais e indicadores de Business Intelligence (BI), conectando diretamente a teoria dos sistemas de informação às demandas estratégicas de mercado.

1.3 Aspectos Metodológicos

O presente estudo caracteriza-se como uma pesquisa aplicada, de natureza quantitativa e qualitativa, voltada à resolução de um problema prático de engenharia de dados. Para a concepção do trabalho, fez-se uso inicial da pesquisa bibliográfica, fundamentada em literatura de referência sobre modelagem relacional, normalização de dados e linguagem SQL, servindo de base para o referencial teórico.

A parte prática seguiu o fluxo de desenvolvimento de sistemas de informação, dividido em quatro etapas fundamentais:

1. **Análise de Requisitos e Regras de Negócio:** Mapeamento do fluxo de um e-commerce de roupas para a definição das entidades e restrições de integridade;
2. **Modelagem de Dados:** Elaboração do esquema lógico e conceitual por meio da ferramenta dbdiagram.io, utilizando a sintaxe DBML (Database Markup Language) para garantir o mapeamento das chaves primárias e estrangeiras;
3. **Implementação Física e Carga de Dados:** Construção do banco de dados (DDL) e automação da carga transacional de 1.000 pedidos (DML) via rotinas programáticas em T-SQL no Microsoft SQL Server;
4. **Validação e Análise Analítica:** Desenvolvimento e execução de 30 consultas SQL avançadas para auditoria de dados e extração de relatórios gerenciais, cujos resultados serão detalhados nos capítulos subsequentes deste documento.

1.4 Aporte Teórico

O embasamento teórico deste trabalho sustenta-se nos princípios do Modelo Relacional, consolidado por autores de referência como Elmasri e Navathe (2011) e Date (2004). Esse modelo fornece as diretrizes fundamentais para a estruturação de dados em tabelas (relações), garantindo a consistência lógica por meio de restrições de integridade e do mapeamento estrito de chaves primárias e estrangeiras.

Complementarmente, utiliza-se a Teoria da Normalização, com foco nas três primeiras formas normais descritas por Heuser (2009), indispensáveis para a eliminação de redundâncias e prevenção de anomalias operacionais em ambientes transacionais (OLTP). A transposição desse arcabouço conceitual para o nível físico baseia-se na especificação da linguagem Transact-SQL (T-SQL) dentro do ecossistema do Microsoft SQL Server, cuja literatura técnica orienta o controle de concorrência, a preservação das propriedades ACID e a otimização de consultas analíticas complexas.

2 FUNDAMENTAÇÃO TEÓRICA

O desenvolvimento de um sistema de gerenciamento de banco de dados corporativo exige um embasamento sólido em teorias de estruturação da informação. Neste capítulo, são apresentados os conceitos fundamentais que norteiam a arquitetura de dados proposta, abordando o modelo relacional, as regras de normalização e a análise de sistemas análogos no contexto do comércio eletrônico.

2.1 O Modelo Relacional e as Restrições de Integridade

O modelo relacional, consolidado por autores clássicos da ciência da computação como Elmasri e Navathe (2011), representa a informação por meio de coleções de tabelas bidimensionais (relações). A integridade desse modelo repousa sobre a definição rigorosa de chaves primárias (PK) e estrangeiras (FK), que garantem, respectivamente, a unicidade dos registros e a consistência das associações entre diferentes entidades. No contexto de operações comerciais online, essa arquitetura estruturada no nível do Sistema de Gerenciamento de Banco de Dados (SGBD) é vital para processar transações simultâneas de clientes e pedidos sem comprometer as propriedades ACID (Atomicidade, Consistência, Isolamento e Durabilidade) do ecossistema.

2.2 Normalização de Dados

A normalização é uma técnica de projeto de banco de dados essencial para mitigar redundâncias e prevenir anomalias de inserção, atualização e exclusão. Conforme descrito por Heuser (2009), o processo consiste na aplicação de formas normais sequenciais baseadas em dependências funcionais. Para assegurar o desempenho e a confiabilidade de um banco de dados transacional (OLTP), a estrutura arquitetada deve atingir a Terceira Forma Normal (3FN). Isso determina que todos os atributos não chave devem depender inteira e exclusivamente da chave primária da tabela, regra técnica que foi rigorosamente aplicada na modelagem do catálogo de produtos e no detalhamento do carrinho de compras deste projeto.

3 PROJETO PROPOSTO

Nesta seção são apresentadas detalhadamente a metodologia utilizada para a concepção do banco de dados do e-commerce de vestuário, detalhando as escolhas arquiteturais, o mapeamento de requisitos, as restrições de integridade e as estratégias empregadas na implementação física e na carga automatizada de dados.

3.1 Considerações Iniciais

O projeto foi concebido para simular o ambiente de persistência de uma plataforma de vendas online de escala corporativa. A engenharia do banco de dados seguiu um fluxo rigoroso focado em garantir a consistência das informações operacionais e o isolamento transacional, servindo como infraestrutura ideal para auditorias complexas e geração de relatórios gerenciais confiáveis.

3.2 Requisitos e Regras de Negócio

O sistema foi projetado sob um conjunto de restrições operacionais estritas para mitigar falhas comuns no ecossistema de varejo digital. As principais diretrizes estabelecidas foram:

- Cada cliente possui um registro único e obrigatório indexado por um CPF válido e e-mail exclusivo;
- O catálogo de produtos gerencia variações multidimensionais de SKU (Unidade de Manutenção de Estoque), contemplando tamanho e cor de forma independente para cada item;
- Um pedido pode conter múltiplos produtos distintos, gravando o preço unitário histórico da peça no exato momento da venda para evitar distorções financeiras em caso de reajustes futuros no catálogo;
- A liquidação financeira possui relacionamento exclusivo e obrigatório com o cabeçalho da compra, gerando um registro de pagamento apenas para pedidos que assumam os estados de "Concluído" ou "Enviado".

Tabela 1 – Dicionário de Dados: CLIENTE

Atributo	Tipo SQL	Restrições	Descrição
ID_CLIENTE	INT	PK, IDENTITY(1,1)	Identificador único do cliente.
NOME	VARCHAR(100)	NOT NULL	Nome completo do cliente.
EMAIL	VARCHAR(100)	UNIQUE, NOT NULL	Endereço de e-mail exclusivo.
CPF	CHAR(11)	UNIQUE, NOT NULL	Documento de pessoa física (11 dígitos numéricos).
DATA_CADASTRO	DATETIME	DEFAULT GETDATE()	Data e hora do registro no sistema.
ESTADO_UF	CHAR(2)	NOT NULL	Sigla do estado de residência (ex: SP, MG).

Tabela 2 – Dicionário de Dados: CATEGORIA

Atributo	Tipo SQL	Restrições	Descrição
ID_CATEGORIA	INT	PK, IDENTITY(1,1)	Identificador único da categoria.

NOME_CATEGORIA	VARCHAR(50)	NOT NULL	Nome da categoria (ex: Camisetas, Acessórios).
DESCRICAO	VARCHAR(255)	NULL	Descrição detalhada dos tipos de itens.

Tabela 3 – Dicionário de Dados: PRODUTO

Atributo	Tipo SQL	Restrições	Descrição
ID_PRODUTO	INT	PK, IDENTITY(1,1)	Identificador único do produto no catálogo.
NOME_PRODUTO	VARCHAR(100)	NOT NULL	Nome comercial da peça.
TAMANHO	VARCHAR(10)	NULL	Tamanho da peça (ex: P, M, G, 40).
COR	VARCHAR(30)	NULL	Cor predominante do item.
PRECO_ATUAL	DECIMAL(10,2)	NOT NULL	Preço de venda atual vigente na loja.

QTD_ESTOQUE	INT	NOT NULL	Quantidade de unidades físicas disponíveis.
ID_CATEGORIA	INT	FK, NOT NULL	Chave estrangeira referenciando a Categoria.

Tabela 4 – Dicionário de Dados: PEDIDO

Atributo	Tipo SQL	Restrições	Descrição
ID_PEDIDO	INT	PK, IDENTITY(1,1)	Identificador único do cabeçalho do pedido.
DATA_PEDIDO	DATETIME	DEFAULT GETDATE()	Data e hora em que o carrinho foi fechado.
STATUS_PEDIDO	VARCHAR(30)	NOT NULL	Situação atual (ex: Concluído, Pendente, Cancelado).
ID_CLIENTE	INT	FK, NOT NULL	Chave estrangeira referenciando o Cliente comprador.

Tabela 5 – Dicionário de Dados: ITEM_PEDIDO

Atributo	Tipo SQL	Restrições	Descrição
ID_ITEM	INT	PK, IDENTITY(1,1)	Identificador único da linha do carrinho.
ID_PEDIDO	INT	FK, NOT NULL	Chave estrangeira referenciando o Pedido.
ID_PRODUTO	INT	FK, NOT NULL	Chave estrangeira referenciando o Produto.
QUANTIDADE	INT	NOT NULL	Quantidade de peças compradas deste produto.
PRECO_UNIT_VENDA	DECIMAL(10,2)	NOT NULL	Preço unitário congelado no momento da venda.

Tabela 6 – Dicionário de Dados: PAGAMENTO

Atributo	Tipo SQL	Restrições	Descrição
ID_PAGAMENTO	INT	PK, IDENTITY(1,1)	Identificador único do registro financeiro.
ID_PEDIDO	INT	FK, UNIQUE, NOT NULL	Chave estrangeira para o Pedido (relacionamento 1:1).
METODO	VARCHAR(30)	NOT NULL	Forma utilizada (ex: PIX, Cartão de Crédito).
DATA_PAGAMENTO	DATETIME	NOT NULL	Data e hora exata da liquidação financeira.
VALOR_PAGO	DECIMAL(10,2)	NOT NULL	Valor financeiro total efetivamente processado.

3.3 Modelagem Lógica e Estrutura de Dados

A arquitetura lógica foi desenhada por meio da ferramenta dbdiagram.io, utilizando a sintaxe declarativa DBML (Database Markup Language). O modelo resultante foi normalizado até a Terceira Forma Normal (3FN), segmentando o domínio em 6 entidades interdependentes: CLIENTE, CATEGORIA, PRODUTO, PEDIDO, ITEM_PEDIDO e PAGAMENTO. Essa separação impede o acoplamento estrutural e elimina anomalias decorrentes da redundância de dados, assegurando que cada fato seja armazenado em apenas um lugar do esquema.

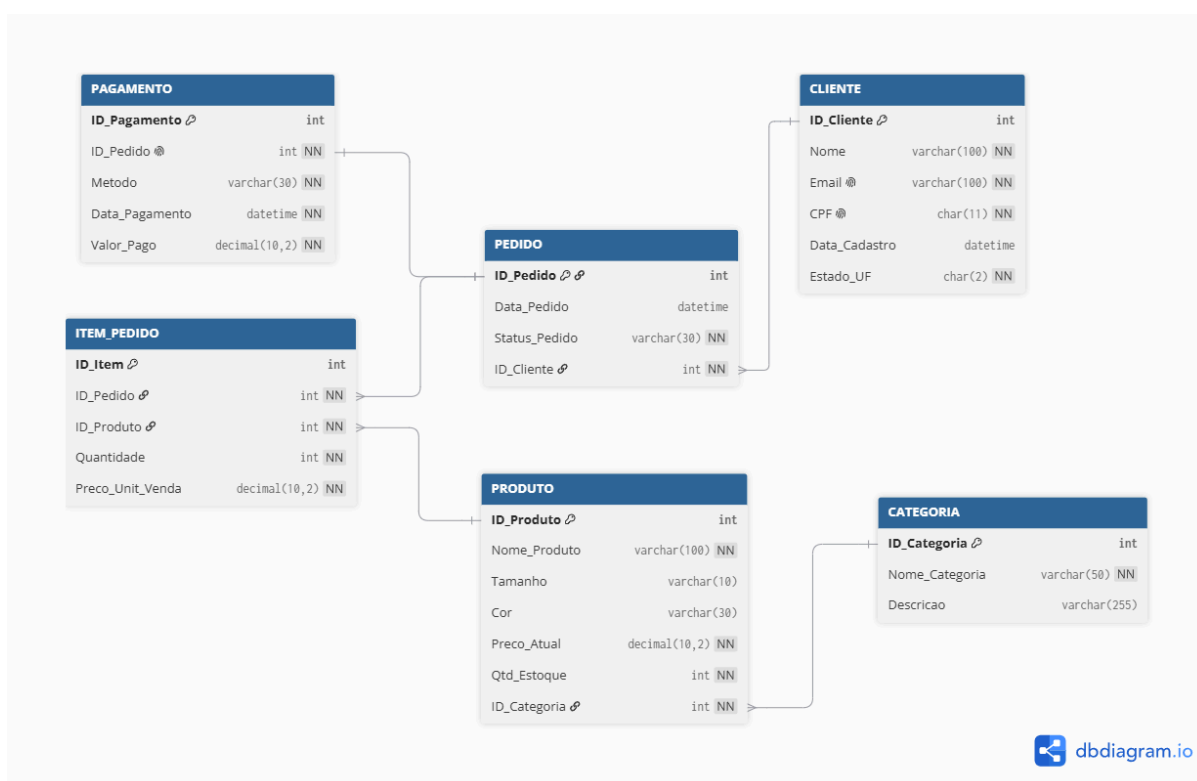


Figura 1 – Modelo Lógico do Banco de Dados

3.4 Implementação Física (DDL)

A transposição do modelo lógico para o ambiente físico foi realizada no SGBD Microsoft SQL Server. O script de definição de dados (DDL) a seguir detalha a criação estrutural do banco de dados, estabelecendo as tipagens corretas (como DECIMAL para valores monetários e DATETIME para registros temporais) e as restrições estruturais de NOT NULL, UNIQUE e integridade referencial (Foreign Keys).

-- =====

-- SCRIPT DDL: CRIAÇÃO DO BANCO E TABELAS

-- =====

CREATE DATABASE DB_ECOMMERCE_VESTUARIO;

GO

USE DB_ECOMMERCE_VESTUARIO;

GO

-- Criação da Tabela CLIENTE

CREATE TABLE CLIENTE (

 ID_CLIENTE INT IDENTITY(1,1) PRIMARY KEY,

 NOME VARCHAR(100) NOT NULL,

 EMAIL VARCHAR(100) UNIQUE NOT NULL,

 CPF CHAR(11) UNIQUE NOT NULL,

 DATA_CADASTRO DATETIME DEFAULT GETDATE(),

 ESTADO_UF CHAR(2) NOT NULL

);

-- Criação da Tabela CATEGORIA

CREATE TABLE CATEGORIA (

 ID_CATEGORIA INT IDENTITY(1,1) PRIMARY KEY,

 NOME_CATEGORIA VARCHAR(50) NOT NULL,

 DESCRICAO VARCHAR(255) NULL

);

-- Criação da Tabela PRODUTO

CREATE TABLE PRODUTO (

 ID_PRODUTO INT IDENTITY(1,1) PRIMARY KEY,

 NOME_PRODUTO VARCHAR(100) NOT NULL,

 TAMANHO VARCHAR(10) NULL,

 COR VARCHAR(30) NULL,

 PRECO_ATUAL DECIMAL(10,2) NOT NULL,

 QTD_ESTOQUE INT NOT NULL,

 ID_CATEGORIA INT NOT NULL,

```
    CONSTRAINT FK_PRODUTO_CATEGORIA FOREIGN KEY (ID_CATEGORIA)
REFERENCES CATEGORIA(ID_CATEGORIA)
);
```

-- Criação da Tabela PEDIDO

```
CREATE TABLE PEDIDO (
    ID_PEDIDO INT IDENTITY(1,1) PRIMARY KEY,
    DATA_PEDIDO DATETIME DEFAULT GETDATE(),
    STATUS_PEDIDO VARCHAR(30) NOT NULL,
    ID_CLIENTE INT NOT NULL,
    CONSTRAINT FK_PEDIDO_CLIENTE FOREIGN KEY (ID_CLIENTE)
REFERENCES CLIENTE(ID_CLIENTE)
);
```

-- Criação da Tabela ITEM_PEDIDO

```
CREATE TABLE ITEM_PEDIDO (
    ID_ITEM INT IDENTITY(1,1) PRIMARY KEY,
    ID_PEDIDO INT NOT NULL,
    ID_PRODUTO INT NOT NULL,
    QUANTIDADE INT NOT NULL,
    PRECO_UNIT_VENDA DECIMAL(10,2) NOT NULL,
    CONSTRAINT FK_ITEM_PEDIDO FOREIGN KEY (ID_PEDIDO) REFERENCES
PEDIDO(ID_PEDIDO),
    CONSTRAINT FK_ITEM_PRODUTO FOREIGN KEY (ID_PRODUTO)
REFERENCES PRODUTO(ID_PRODUTO)
);
```

-- Criação da Tabela PAGAMENTO

```
CREATE TABLE PAGAMENTO (
    ID_PAGAMENTO INT IDENTITY(1,1) PRIMARY KEY,
    ID_PEDIDO INT NOT NULL UNIQUE,
    METODO VARCHAR(30) NOT NULL,
    DATA_PAGAMENTO DATETIME NOT NULL,
    VALOR_PAGO DECIMAL(10,2) NOT NULL,
```

```

        CONSTRAINT FK_PAGAMENTO_PEDIDO FOREIGN KEY (ID_PEDIDO)
REFERENCES PEDIDO(ID_PEDIDO)
);

```

3.5 Mecanismo de Carga e Injeção Massiva (DML)

Para simular a volumetria de um ambiente de produção real, evitou-se a inserção manual de pedidos. Implementou-se um algoritmo programático em T-SQL que insere o catálogo de produtos, clientes base e utiliza estruturas de controle (WHILE), sorteio probabilístico (NEWID) e tabelas temporárias para garantir a injeção síncrona e automatizada de 1.000 pedidos. O comando SET DATEFORMAT ymd foi utilizado para blindar a inserção contra erros de localidade.

```

--
=====
=
-- SCRIPT DML: CARGA DE DADOS COMPLETA E BLINDADA
--
=====
=
USE DB_ECOMMERCE_VESTUARIO;
GO

SET DATEFORMAT ymd;
GO

DELETE FROM PAGAMENTO;
DELETE FROM ITEM_PEDIDO;
DELETE FROM PEDIDO;
DELETE FROM PRODUTO;
DELETE FROM CATEGORIA;
DELETE FROM CLIENTE;

```

```

DBCC CHECKIDENT ('PAGAMENTO', RESEED, 0);
DBCC CHECKIDENT ('ITEM_PEDIDO', RESEED, 0);
DBCC CHECKIDENT ('PEDIDO', RESEED, 0);
DBCC CHECKIDENT ('PRODUTO', RESEED, 0);
DBCC CHECKIDENT ('CATEGORIA', RESEED, 0);
DBCC CHECKIDENT ('CLIENTE', RESEED, 0);
GO

```

```
--
```

```
=====
```

```
=
```

```
-- 2. CARGA DE CLIENTES (100 Registros)
```

```
--
```

```
=====
```

```
=
```

```
INSERT INTO CLIENTE (NOME, EMAIL, CPF, ESTADO_UF, DATA_CADASTRO)
VALUES
```

```
('Lucas Almeida', 'lucas.almeida1@email.com', '48291038472', 'SP', '2023-06-15
10:30:00'),
```

```
('Mariana Costa', 'mariana.costa2@email.com', '83920193847', 'RJ', '2024-02-20
14:15:00'),
```

```
('Pedro Santos', 'pedro.santos3@email.com', '10394827364', 'MG', '2025-11-05
09:45:00'),
```

```
('Ana Oliveira', 'ana.oliveira4@email.com', '58392019283', 'PR', '2023-08-12
16:20:00'),
```

```
('Rafael Souza', 'rafael.souza5@email.com', '74839201928', 'RS', '2026-01-10
11:05:00'),
```

```
('Beatriz Lima', 'beatriz.lima6@email.com', '29384756382', 'SC', '2024-07-22
08:30:00'),
```

```
('Thiago Silva', 'thiago.silva7@email.com', '84930291827', 'SP', '2025-03-18
15:50:00'),
```

```
('Camila Pereira', 'camila.pereira8@email.com', '19283746573', 'RJ', '2023-11-30
19:10:00'),
```

('Felipe Carvalho', 'felipe.carvalho9@email.com', '57483920192', 'MG', '2026-04-02 13:25:00'),
('Juliana Rodrigues', 'juliana.rodrig10@email.com', '93847562839', 'SP', '2024-09-14 17:40:00'),
('Bruno Gomes', 'bruno.gomes11@email.com', '48573920193', 'PR', '2025-06-25 10:15:00'),
('Letícia Martins', 'leticia.mart12@email.com', '10293847562', 'RS', '2023-12-08 20:05:00'),
('Gabriel Ferreira', 'gabriel.ferr13@email.com', '67584930219', 'SC', '2024-05-19 12:35:00'),
('Fernanda Alves', 'fernanda.alves14@email.com', '28394051627', 'SP', '2025-08-07 14:55:00'),
('Diego Rocha', 'diego.rocha15@email.com', '84950392817', 'RJ', '2026-02-28 09:10:00'),
('Amanda Ribeiro', 'amanda.rib16@email.com', '39485726153', 'MG', '2023-07-03 16:45:00'),
('Marcelo Dias', 'marcelo.dias17@email.com', '75849302918', 'PR', '2024-11-11 11:20:00'),
('Jessica Castro', 'jessica.cast18@email.com', '10293847563', 'SP', '2025-01-29 18:30:00'),
('Rodrigo Mendes', 'rodrigo.men19@email.com', '58493021928', 'RS', '2026-05-15 15:05:00'),
('Larissa Nunes', 'larissa.nun20@email.com', '93847561029', 'SC', '2023-09-21 10:40:00'),
('Guilherme Barbosa', 'guilherme.bar21@email.com', '20394857162', 'SP', '2024-04-06 14:00:00'),
('Carolina Pinto', 'carolina.pin22@email.com', '67584930291', 'RJ', '2025-10-12 08:50:00'),
('Leonardo Farias', 'leonardo.far23@email.com', '19283746501', 'MG', '2023-10-05 17:15:00'),
('Patricia Moura', 'patricia.mou24@email.com', '48573920182', 'SP', '2026-03-20 12:45:00'),
('Eduardo Vieira', 'eduardo.vie25@email.com', '83920192837', 'PR', '2024-08-30 09:55:00'),

('Renata Cavalcanti', 'renata.caval26@email.com', '57483920183', 'RS', '2025-02-14 16:35:00'),
('Vitor Cardoso', 'vitor.card27@email.com', '10293847504', 'SC', '2023-05-29 11:10:00'),
('Daniela Monteiro', 'daniela.mon28@email.com', '93847562810', 'SP', '2024-12-05 14:20:00'),
('Alexandre Teixeira', 'alexandre.tei29@email.com', '28394051638', 'RJ', '2025-09-22 10:05:00'),
('Aline Moraes', 'aline.mor30@email.com', '67584930202', 'MG', '2026-04-18 16:50:00'),
('Matheus Correia', 'matheus.cor31@email.com', '19283746512', 'SP', '2023-08-01 09:15:00'),
('Sabrina Ramos', 'sabrina.ram32@email.com', '84950392828', 'PR', '2024-03-10 13:40:00'),
('André Batista', 'andre.bati33@email.com', '39485726164', 'RS', '2025-07-28 11:25:00'),
('Vanessa Borges', 'vanessa.bor34@email.com', '75849302929', 'SC', '2026-05-25 15:35:00'),
('Igor Machado', 'igor.mac35@email.com', '10293847574', 'SP', '2023-06-19 18:00:00'),
('Marcos Souza', 'marcos.souza.inativo@email.com', '29481039582', 'SP', '2023-07-10 11:00:00'),
('Aline Cavalcante', 'aline.cav.inativo@email.com', '74839102948', 'RJ', '2024-01-15 14:30:00'),
('Ricardo Santos', 'ricardo.santos.inativo@email.com', '10394827193', 'MG', '2024-08-22 09:15:00'),
('Patricia Lima', 'patricia.lima.inativo@email.com', '58392019482', 'PR', '2025-03-05 16:00:00'),
('Fabio Oliveira', 'fabio.oli.inativo@email.com', '84930291573', 'SC', '2025-11-12 10:45:00'),
('Tatiana Campos', 'tatiana.cam36@email.com', '58493021939', 'RJ', '2024-10-08 10:45:00'),
('Caio Freitas', 'caio.frei37@email.com', '93847561040', 'MG', '2025-04-20 14:10:00'),
('Erica Pires', 'erica.pir38@email.com', '20394857173', 'SP', '2026-01-05 09:55:00'),

('Ricardo Duarte', 'ricardo.dua39@email.com', '67584930213', 'PR', '2023-11-12 16:30:00'),
('Mônica Nogueira', 'monica.nog40@email.com', '19283746523', 'RS', '2024-06-21 11:15:00'),
('Leandro Barros', 'leandro.bar41@email.com', '48573920104', 'SC', '2025-12-03 13:50:00'),
('Cintia Aragão', 'cintia.ara42@email.com', '83920192848', 'SP', '2023-05-30 08:40:00'),
('Renato Pacheco', 'renato.pac43@email.com', '57483920194', 'RJ', '2024-01-15 15:20:00'),
('Flavia Cruz', 'flavia.cru44@email.com', '10293847515', 'MG', '2025-05-29 10:05:00'),
('Marcos Viana', 'marcos.via45@email.com', '93847562821', 'SP', '2026-02-14 17:35:00'),
('Priscila Guedes', 'priscila.gue46@email.com', '28394051649', 'PR', '2023-09-02 12:10:00'),
('Tiago Lemos', 'tiago.lem47@email.com', '67584930224', 'RS', '2024-04-25 09:45:00'),
('Bruna Peixoto', 'bruna.pei48@email.com', '19283746534', 'SC', '2025-10-18 14:30:00'),
('Douglas Neves', 'douglas.nev49@email.com', '84950392839', 'SP', '2026-03-05 11:20:00'),
('Gisele Sales', 'gisele.sal50@email.com', '39485726175', 'RJ', '2023-12-20 16:15:00'),
('Roberto Dantas', 'roberto.dan51@email.com', '75849302940', 'MG', '2024-07-09 10:50:00'),
('Bianca Fonseca', 'bianca.fon52@email.com', '10293847585', 'SP', '2025-02-22 15:05:00'),
('Fernando Braga', 'fernando.bra53@email.com', '58493021950', 'PR', '2023-08-14 08:30:00'),
('Luciana Tavares', 'luciana.tav54@email.com', '93847561051', 'RS', '2024-11-28 13:40:00'),
('Gustavo Melo', 'gustavo.mel55@email.com', '20394857184', 'SC', '2025-06-11 17:25:00'),

('Thais Assis', 'thais.ass56@email.com', '67584930235', 'SP', '2026-04-30 09:10:00'),
('Julio Xavier', 'julio.xav57@email.com', '19283746545', 'RJ', '2023-10-25 14:55:00'),
('Kelly Siqueira', 'kelly.siq58@email.com', '48573920115', 'MG', '2024-05-07
11:20:00'),
('Fabio Rezende', 'fabio.rez59@email.com', '83920192859', 'SP', '2025-12-19
16:35:00'),
('Marina Diniz', 'marina.din60@email.com', '57483920205', 'PR', '2023-06-02
10:05:00'),
('Henrique Lira', 'henrique.lir61@email.com', '10293847526', 'RS', '2024-02-18
15:50:00'),
('Dayane Soares', 'dayane.soa62@email.com', '93847562832', 'SC', '2025-09-04
08:45:00'),
('Wesley Cordeiro', 'wesley.cor63@email.com', '28394051660', 'SP', '2026-01-22
13:10:00'),
('Raquel Farias', 'raquel.far64@email.com', '67584930246', 'RJ', '2023-07-16
17:40:00'),
('Samuel Prado', 'samuel.pra65@email.com', '19283746556', 'MG', '2024-10-29
10:15:00'),
('Clara Leite', 'clara.lei66@email.com', '84950392850', 'SP', '2025-04-12 14:55:00'),
('Arthur Maciel', 'arthur.mac67@email.com', '39485726186', 'PR', '2023-11-25
09:30:00'),
('Laura Bentes', 'laura.ben68@email.com', '75849302951', 'RS', '2024-08-08
16:20:00'),
('Joao Figueiredo', 'joao.fig69@email.com', '10293847596', 'SC', '2025-03-01
11:05:00'),
('Alice Marinho', 'alice.mar70@email.com', '58493021961', 'SP', '2026-05-10
15:40:00'),
('Victor Bessa', 'victor.bes71@email.com', '93847561062', 'RJ', '2023-09-15
08:50:00'),
('Sofia Esteves', 'sofia.est72@email.com', '20394857195', 'MG', '2024-12-30
13:25:00'),
('Danilo Guerra', 'danilo.gue73@email.com', '67584930257', 'SP', '2025-07-23
17:15:00'),
('Julia Barreto', 'julia.bar74@email.com', '19283746567', 'PR', '2026-02-04 10:45:00'),

('Marcio Antunes', 'marcio.ant75@email.com', '48573920126', 'RS', '2023-05-31 15:10:00'),
('Melissa Godoy', 'melissa.god76@email.com', '83920192870', 'SC', '2024-03-24 09:55:00'),
('Alex Faria', 'alex.far77@email.com', '57483920216', 'SP', '2025-10-06 14:30:00'),
('Natalia Paiva', 'natalia.pai78@email.com', '10293847537', 'RJ', '2026-04-19 11:20:00'),
('Erick Toledo', 'erick.tol79@email.com', '93847562843', 'MG', '2023-08-27 16:05:00'),
('Suelen Viana', 'suelen.via80@email.com', '28394051671', 'SP', '2024-06-10 10:50:00'),
('Hugo Muniz', 'hugo.mun81@email.com', '67584930268', 'PR', '2025-01-02 15:35:00'),
('Paloma Brito', 'paloma.bri82@email.com', '19283746578', 'RS', '2023-12-14 08:40:00'),
('Valdir Santana', 'valdir.san83@email.com', '84950392861', 'SC', '2024-09-27 13:15:00'),
('Eliane Marques', 'eliane.mar84@email.com', '39485726197', 'SP', '2025-05-11 17:50:00'),
('Cristiano Ferraz', 'cristiano.fer85@email.com', '75849302962', 'RJ', '2026-03-28 09:25:00'),
('Janaina Lins', 'janaina.lin86@email.com', '10293847607', 'MG', '2023-10-18 14:10:00'),
('William Souza', 'william.sou87@email.com', '58493021972', 'SP', '2024-05-02 11:55:00'),
('Milena Furtado', 'milena.fur88@email.com', '93847561073', 'PR', '2025-11-20 16:45:00'),
('Vagner Nogueira', 'vagner.nog89@email.com', '20394857206', 'RS', '2023-07-05 10:30:00'),
('Diana Machado', 'diana.mac90@email.com', '67584930279', 'SC', '2024-11-14 15:05:00'),
('Cesar Bueno', 'cesar.bue91@email.com', '19283746589', 'SP', '2025-08-03 08:50:00'),
('Livia Dutra', 'livia.dut92@email.com', '48573920137', 'RJ', '2026-05-20 13:20:00'),

```
('Renan Gusmão', 'renan.gus93@email.com', '83920192881', 'MG', '2023-06-28
17:40:00'),
('Miriam Telles', 'miriam.tel94@email.com', '57483920227', 'SP', '2024-04-11
10:15:00'),
('Viviane Matos', 'viviane.mat100@email.com', '84950392872', 'MG', '2024-10-22
08:35:00');
GO
```

```
-- Carga de Categorias
```

```
INSERT INTO CATEGORIA (NOME_CATEGORIA, DESCRICAO) VALUES
('Camisetas', 'Camisetas básicas, estampadas e oversized'),
('Calças e Bermudas', 'Jeans, sarja, alfaiataria e moletom'),
('Casacos e Jaquetas', 'Jaquetas de couro, jeans e corta-vento'),
('Acessórios', 'Cintos, carteiras, bonés e óculos'),
('Moda Íntima', 'Cuecas e pijamas masculinos');
GO
```

```
--
```

```
=====
```

```
=
```

```
-- 4. CARGA DE PRODUTOS (30 Registros Estratégicos com SKUs)
```

```
--
```

```
=====
```

```
=
```

```
INSERT INTO PRODUTO (NOME_PRODUTO, TAMANHO, COR, PRECO_ATUAL,
QTD_ESTOQUE, ID_CATEGORIA) VALUES
```

```
-- CATEGORIA 1: Camisetas
```

```
('Camiseta Básica Algodão', 'P', 'Branca', 59.90, 100, 1),
('Camiseta Básica Algodão', 'M', 'Branca', 59.90, 150, 1),
('Camiseta Básica Algodão', 'G', 'Branca', 59.90, 80, 1),
('Camiseta Oversized Street', 'M', 'Preta', 89.90, 40, 1),
('Camiseta Oversized Street', 'G', 'Preta', 89.90, 0, 1),
('Camiseta Polo Piquet', 'M', 'Azul Marinho', 119.90, 60, 1),
```

-- CATEGORIA 2: Calças e Bermudas

('Calça Jeans Slim', '38', 'Azul Escuro', 179.90, 50, 2),
 ('Calça Jeans Slim', '40', 'Azul Escuro', 179.90, 70, 2),
 ('Calça Jeans Slim', '42', 'Azul Escuro', 179.90, 30, 2),
 ('Calça Sarja Cargo', '40', 'Bege', 199.90, 45, 2),
 ('Calça Sarja Cargo', '42', 'Bege', 199.90, 20, 2),
 ('Bermuda Moletom Confort', 'M', 'Cinza Mescla', 119.90, 100, 2),
 ('Bermuda Moletom Confort', 'G', 'Cinza Mescla', 119.90, 85, 2),

-- CATEGORIA 3: Casacos e Jaquetas

('Jaqueta Couro Sintético', 'M', 'Preta', 349.90, 15, 3),
 ('Jaqueta Couro Sintético', 'G', 'Preta', 349.90, 5, 3),
 ('Jaqueta Corta-Vento', 'M', 'Preto/Refletivo', 189.90, 40, 3),
 ('Jaqueta Corta-Vento', 'G', 'Preto/Refletivo', 189.90, 25, 3),
 ('Casaco Moletom Canguru', 'M', 'Bordô', 159.90, 60, 3),
 ('Casaco Moletom Canguru', 'G', 'Bordô', 159.90, 0, 3),

-- CATEGORIA 4: Acessórios

('Cinto Couro Dupla Face', 'único', 'Preto/Marrom', 79.90, 120, 4),
 ('Carteira Slim Couro', 'único', 'Café', 89.90, 90, 4),
 ('Boné Trucker Aba Curva', 'único', 'Preto', 69.90, 0, 4),
 ('Boné Trucker Aba Curva', 'único', 'Verde Militar', 69.90, 35, 4),
 ('Óculos de Sol Aviador', 'único', 'Dourado', 149.90, 20, 4),
 ('Kit 3 Pares Meia Cano Alto', 'único', 'Branca', 49.90, 200, 4),

-- CATEGORIA 5: Moda íntima

('Kit 3 Cuecas Boxer', 'M', 'Sortido', 89.90, 150, 5),
 ('Kit 3 Cuecas Boxer', 'G', 'Sortido', 89.90, 180, 5),
 ('Kit 3 Cuecas Boxer', 'GG', 'Sortido', 89.90, 110, 5),
 ('Samba-Canção Algodão', 'M', 'Azul Xadrez', 55.90, 60, 5),
 ('Samba-Canção Algodão', 'G', 'Azul Xadrez', 55.90, 45, 5);

GO

```

--
=====
=
-- GERAÇÃO EM MASSA: 1.000 PEDIDOS E PAGAMENTOS (T-SQL CORRIGIDO)
--
=====
=
DECLARE @TotalPedidos INT = 1000;
DECLARE @Contador INT = 1;

DECLARE @ID_Cliente INT;
DECLARE @DataPedido DATETIME;
DECLARE @Status VARCHAR(30);
DECLARE @ID_Pedido_Gerado INT;
DECLARE @NumItens INT;
DECLARE @ItemAtual INT;
DECLARE @TotalCarrinho DECIMAL(10,2);
DECLARE @Metodo VARCHAR(30);

SET NOCOUNT ON;

WHILE @Contador <= @TotalPedidos
BEGIN
    SET @ID_Cliente = (SELECT TOP 1 ID_CLIENTE FROM CLIENTE WHERE
ID_CLIENTE <= 100 ORDER BY NEWID());
    SET @DataPedido = DATEADD(DAY, -(ABS(CHECKSUM(NEWID())) % 1000),
GETDATE());

    DECLARE @RandStatus INT = ABS(CHECKSUM(NEWID())) % 100;
    SET @Status = CASE
        WHEN @RandStatus < 70 THEN 'Concluído'
        WHEN @RandStatus < 85 THEN 'Enviado'
        WHEN @RandStatus < 95 THEN 'Pendente'
        ELSE 'Cancelado'

```

END;

```
INSERT INTO PEDIDO (DATA_PEDIDO, STATUS_PEDIDO, ID_CLIENTE)
VALUES (@DataPedido, @Status, @ID_Cliente);
```

```
SET @ID_Pedido_Gerado = SCOPE_IDENTITY();
```

```
SET @NumItens = (ABS(CHECKSUM(NEWID())) % 4) + 1;
```

```
SET @ItemAtual = 1;
```

```
SET @TotalCarrinho = 0;
```

```
WHILE @ItemAtual <= @NumItens
```

```
BEGIN
```

```
    DECLARE @TabelaProdutoTemp TABLE (ID INT, Preco DECIMAL(10,2));
```

```
    INSERT INTO @TabelaProdutoTemp
```

```
        SELECT TOP 1 ID_PRODUTO, PRECO_ATUAL FROM PRODUTO ORDER
BY NEWID();
```

```
    DECLARE @Qtd INT = (ABS(CHECKSUM(NEWID())) % 3) + 1;
```

```
    DECLARE @ID_Prod_Sorteado INT = (SELECT TOP 1 ID FROM
@TabelaProdutoTemp);
```

```
    DECLARE @Preco_Prod_Sorteado DECIMAL(10,2) = (SELECT TOP 1 Preco
FROM @TabelaProdutoTemp);
```

```
    INSERT INTO ITEM_PEDIDO (ID_PEDIDO, ID_PRODUTO, QUANTIDADE,
PRECO_UNIT_VENDA)
```

```
        VALUES (@ID_Pedido_Gerado, @ID_Prod_Sorteado, @Qtd,
@Preco_Prod_Sorteado);
```

```
    SET @TotalCarrinho = @TotalCarrinho + (@Qtd * @Preco_Prod_Sorteado);
```

```
    SET @ItemAtual = @ItemAtual + 1;
```

```
    DELETE FROM @TabelaProdutoTemp;
```

END

```
IF @Status IN ('Concluído', 'Enviado')
BEGIN
    DECLARE @RandMetodo INT = ABS(CHECKSUM(NEWID())) % 100;
    SET @Metodo = CASE
        WHEN @RandMetodo < 40 THEN 'PIX'
        WHEN @RandMetodo < 70 THEN 'Cartão de Crédito'
        WHEN @RandMetodo < 90 THEN 'Cartão de Débito'
        ELSE 'Boleto'
    END;

    DECLARE @DataPag DATETIME = DATEADD(MINUTE,
ABS(CHECKSUM(NEWID())) % 1440, @DataPedido);

    INSERT INTO PAGAMENTO (ID_PEDIDO, METODO, DATA_PAGAMENTO,
VALOR_PAGO)
        VALUES (@ID_Pedido_Gerado, @Metodo, @DataPag, @TotalCarrinho);
    END
    SET @Contador = @Contador + 1;
END
SET NOCOUNT OFF;
GO
```

4 AVALIAÇÃO

Neste capítulo são apresentados os resultados práticos da implementação e da injeção de dados. A validação do banco de dados relacional construído deu-se por meio da extração de inteligência de negócios (Business Intelligence), assegurando que o modelo responde a cenários complexos do mundo real.

4.1 Consultas Analíticas (SQL)

Para evidenciar a robustez da arquitetura, elaborou-se um conjunto de 30 consultas analíticas utilizando o dialeto T-SQL. As métricas exploram agregações profundas, junções múltiplas (INNER e LEFT JOINS), Subqueries e funções avançadas de janelamento (Window Functions como RANK, LAG e LEAD), essenciais para extrair insights estratégicos.

-- 1. Faturamento Total Acumulado por Categoria

```
SELECT C.NOME_CATEGORIA, SUM(IP.QUANTIDADE *
IP.PRECO_UNIT_VENDA) AS FATURAMENTO_TOTAL
FROM CATEGORIA C
INNER JOIN PRODUTO P ON C.ID_CATEGORIA = P.ID_CATEGORIA
INNER JOIN ITEM_PEDIDO IP ON P.ID_PRODUTO = IP.ID_PRODUTO
INNER JOIN PEDIDO PD ON IP.ID_PEDIDO = PD.ID_PEDIDO
WHERE PD.STATUS_PEDIDO IN ('Concluído', 'Enviado')
GROUP BY C.NOME_CATEGORIA ORDER BY FATURAMENTO_TOTAL DESC;
```

-- 2. Ranking dos 5 Clientes com Maior Volume de Compras

```
SELECT TOP 5 C.NOME, C.ESTADO_UF, COUNT(DISTINCT PD.ID_PEDIDO) AS
QTD_PEDIDOS,
SUM(PG.VALOR_PAGO) AS TOTAL_GASTO, SUM(PG.VALOR_PAGO) /
COUNT(DISTINCT PD.ID_PEDIDO) AS TICKET_MEDIO
FROM CLIENTE C
INNER JOIN PEDIDO PD ON C.ID_CLIENTE = PD.ID_CLIENTE
INNER JOIN PAGAMENTO PG ON PD.ID_PEDIDO = PG.ID_PEDIDO
GROUP BY C.NOME, C.ESTADO_UF ORDER BY TOTAL_GASTO DESC;
```

-- 3. Produtos com Estoque Crítico


```

SELECT NOME_PRODUTO, QTD_ESTOQUE, PRECO_ATUAL, (20 -
QTD_ESTOQUE) * PRECO_ATUAL AS RECEITA_PERDIDA
FROM PRODUTO WHERE QTD_ESTOQUE < 20 ORDER BY QTD_ESTOQUE
ASC;

```

-- 4. Taxa de Rejeição: Percentual de Cancelados

```

WITH MetricasPedidos AS (
    SELECT COUNT(*) AS TOTAL_PEDIDOS, SUM(CASE WHEN STATUS_PEDIDO
= 'Cancelado' THEN 1 ELSE 0 END) AS PEDIDOS_CANCELADOS
    FROM PEDIDO
)
SELECT TOTAL_PEDIDOS, PEDIDOS_CANCELADOS,
CAST((PEDIDOS_CANCELADOS * 100.0 / TOTAL_PEDIDOS) AS DECIMAL(5,2))
AS TAXA_CANCELAMENTO
FROM MetricasPedidos;

```

-- 5. Distribuição de Receita por Método de Pagamento

```

SELECT METODO, COUNT(ID_PAGAMENTO) AS VOL_TRANSACOES,
SUM(VALOR_PAGO) AS RECEITA_PROCESSADA
FROM PAGAMENTO GROUP BY METODO ORDER BY RECEITA_PROCESSADA
DESC;

```

-- 6. Análise de Vendas Cruzadas (Cross-Selling)

```

SELECT PD.ID_PEDIDO, C.NOME, COUNT(IP.ID_PRODUTO) AS
VARIEDADE_ITENS, SUM(IP.QUANTIDADE) AS TOTAL_PECAS
FROM PEDIDO PD
INNER JOIN CLIENTE C ON PD.ID_CLIENTE = C.ID_CLIENTE
INNER JOIN ITEM_PEDIDO IP ON PD.ID_PEDIDO = IP.ID_PEDIDO
GROUP BY PD.ID_PEDIDO, C.NOME, PD.DATA_PEDIDO HAVING
COUNT(IP.ID_PRODUTO) > 3 ORDER BY TOTAL_PECAS DESC;

```

-- 7. Desempenho de Vendas por Estado (UF)

```

SELECT C.ESTADO_UF, COUNT(DISTINCT C.ID_CLIENTE) AS
CLIENTES_ATIVOS, SUM(PG.VALOR_PAGO) AS FATURAMENTO_ESTADUAL

```

```

FROM CLIENTE C
INNER JOIN PEDIDO PD ON C.ID_CLIENTE = PD.ID_CLIENTE
INNER JOIN PAGAMENTO PG ON PD.ID_PEDIDO = PG.ID_PEDIDO
GROUP BY C.ESTADO_UF ORDER BY FATURAMENTO_ESTADUAL DESC;

```

-- 8. Curva de Retenção: Clientes Inativos

```

SELECT C.NOME, DATEDIFF(DAY, C.DATA_CADASTRO, GETDATE()) AS
DIAS_DESDE_CADASTRO
FROM CLIENTE C LEFT JOIN PEDIDO PD ON C.ID_CLIENTE = PD.ID_CLIENTE
WHERE PD.ID_PEDIDO IS NULL ORDER BY DIAS_DESDE_CADASTRO DESC;

```

-- 9. Sazonalidade do Faturamento

```

SELECT YEAR(PG.DATA_PAGAMENTO) AS ANO,
MONTH(PG.DATA_PAGAMENTO) AS MES, SUM(PG.VALOR_PAGO) AS
FATURAMENTO_MENSAL
FROM PAGAMENTO PG GROUP BY YEAR(PG.DATA_PAGAMENTO),
MONTH(PG.DATA_PAGAMENTO) ORDER BY ANO DESC, MES DESC;

```

-- 10. Os 3 Produtos Menos Vendidos

```

SELECT TOP 3 P.NOME_PRODUTO, ISNULL(SUM(IP.QUANTIDADE), 0) AS
TOTAL_VENDIDO
FROM PRODUTO P LEFT JOIN ITEM_PEDIDO IP ON P.ID_PRODUTO =
IP.ID_PRODUTO
GROUP BY P.NOME_PRODUTO ORDER BY TOTAL_VENDIDO ASC;

```

-- 11. Campeões de Venda por Categoria (Window Function: RANK)

```

WITH RankProdutos AS (
    SELECT C.NOME_CATEGORIA, P.NOME_PRODUTO, SUM(IP.QUANTIDADE)
    AS QTD_VENDIDA,
        RANK() OVER(PARTITION BY C.NOME_CATEGORIA ORDER BY
SUM(IP.QUANTIDADE) DESC) AS RANKING
    FROM CATEGORIA C
    INNER JOIN PRODUTO P ON C.ID_CATEGORIA = P.ID_CATEGORIA
    INNER JOIN ITEM_PEDIDO IP ON P.ID_PRODUTO = IP.ID_PRODUTO

```

```

        GROUP BY C.NOME_CATEGORIA, P.NOME_PRODUTO
    )
    SELECT NOME_CATEGORIA, NOME_PRODUTO, QTD_VENDIDA FROM
    RankProdutos WHERE RANKING = 1;

```

-- 12. Cohort de Clientes: Tempo Retido

```

SELECT C.NOME, DATEDIFF(DAY, MIN(PD.DATA_PEDIDO),
MAX(PD.DATA_PEDIDO)) AS DIAS_RETENCAO
FROM CLIENTE C INNER JOIN PEDIDO PD ON C.ID_CLIENTE = PD.ID_CLIENTE
GROUP BY C.NOME HAVING COUNT(PD.ID_PEDIDO) > 1 ORDER BY
DIAS_RETENCAO DESC;

```

-- 13. Faturamento Acumulado Anual (Running Total)

```

WITH FaturamentoMensal AS (
    SELECT YEAR(DATA_PAGAMENTO) AS ANO, MONTH(DATA_PAGAMENTO)
    AS MES, SUM(VALOR_PAGO) AS TOTAL_MES
    FROM PAGAMENTO GROUP BY YEAR(DATA_PAGAMENTO),
    MONTH(DATA_PAGAMENTO)
)
SELECT ANO, MES, TOTAL_MES, SUM(TOTAL_MES) OVER(ORDER BY ANO,
MES ROWS UNBOUNDED PRECEDING) AS FATURAMENTO_ACUMULADO
FROM FaturamentoMensal;

```

-- 14. Pedidos Premium: Acima da Média Nacional

```

SELECT PD.ID_PEDIDO, C.NOME AS CLIENTE, PG.VALOR_PAGO
FROM PEDIDO PD
INNER JOIN CLIENTE C ON PD.ID_CLIENTE = C.ID_CLIENTE
INNER JOIN PAGAMENTO PG ON PD.ID_PEDIDO = PG.ID_PEDIDO
WHERE PG.VALOR_PAGO > (SELECT AVG(VALOR_PAGO) FROM PAGAMENTO)
ORDER BY PG.VALOR_PAGO DESC;

```

-- 15. Matriz de Status de Pedidos por Ano (Pivot)

```

SELECT YEAR(DATA_PEDIDO) AS ANO,

```

```

SUM(CASE WHEN STATUS_PEDIDO = 'Concluído' THEN 1 ELSE 0 END) AS
VOL_CONCLUIDO,
SUM(CASE WHEN STATUS_PEDIDO = 'Cancelado' THEN 1 ELSE 0 END) AS
VOL_CANCELADO
FROM PEDIDO GROUP BY YEAR(DATA_PEDIDO) ORDER BY ANO DESC;

```

-- 16. Capital Imobilizado em Estoque por Categoria

```

SELECT C.NOME_CATEGORIA, SUM(P.QTD_ESTOQUE * P.PRECO_ATUAL) AS
VALOR_IMOBILIZADO
FROM CATEGORIA C INNER JOIN PRODUTO P ON C.ID_CATEGORIA =
P.ID_CATEGORIA
GROUP BY C.NOME_CATEGORIA ORDER BY VALOR_IMOBILIZADO DESC;

```

-- 17. Produtos Órfãos (Nunca Comercializados)

```

SELECT P.NOME_PRODUTO, C.NOME_CATEGORIA
FROM PRODUTO P INNER JOIN CATEGORIA C ON P.ID_CATEGORIA =
C.ID_CATEGORIA
LEFT JOIN ITEM_PEDIDO IP ON P.ID_PRODUTO = IP.ID_PRODUTO WHERE
IP.ID_ITEM IS NULL;

```

-- 18. Perfil Demográfico: Aquisição por Domínio de E-mail

```

SELECT RIGHT(EMAIL, LEN(EMAIL) - CHARINDEX('@', EMAIL)) AS
DOMINIO_EMAIL, COUNT(ID_CLIENTE) AS QTD_CLIENTES
FROM CLIENTE GROUP BY RIGHT(EMAIL, LEN(EMAIL) - CHARINDEX('@',
EMAIL)) ORDER BY QTD_CLIENTES DESC;

```

-- 19. Média de Dias para Recompra (Window Function: LEAD)

```

WITH PedidosCliente AS (
SELECT ID_CLIENTE, DATA_PEDIDO, LEAD(DATA_PEDIDO)
OVER(PARTITION BY ID_CLIENTE ORDER BY DATA_PEDIDO) AS
PROXIMA_COMPRA
FROM PEDIDO
)

```

```

SELECT C.NOME, AVG(DATEDIFF(DAY, PC.DATA_PEDIDO,
PC.PROXIMA_COMPRA)) AS MEDIA_DIAS_RECOMPRA
FROM PedidosCliente PC INNER JOIN CLIENTE C ON PC.ID_CLIENTE =
C.ID_CLIENTE
WHERE PC.PROXIMA_COMPRA IS NOT NULL GROUP BY C.NOME ORDER BY
MEDIA_DIAS_RECOMPRA ASC;

```

-- 20. Market Share Estadual (CROSS JOIN)

```

WITH FaturamentoEstado AS (
    SELECT C.ESTADO_UF, SUM(PG.VALOR_PAGO) AS FATURAMENTO
    FROM CLIENTE C INNER JOIN PEDIDO PD ON C.ID_CLIENTE =
PD.ID_CLIENTE
    INNER JOIN PAGAMENTO PG ON PD.ID_PEDIDO = PG.ID_PEDIDO GROUP
BY C.ESTADO_UF
), FaturamentoTotal AS (SELECT SUM(VALOR_PAGO) AS TOTAL_GERAL FROM
PAGAMENTO)
SELECT FE.ESTADO_UF, CAST((FE.FATURAMENTO * 100.0 / FT.TOTAL_GERAL)
AS DECIMAL(5,2)) AS PERCENTUAL
FROM FaturamentoEstado FE CROSS JOIN FaturamentoTotal FT ORDER BY
PERCENTUAL DESC;

```

-- 21. Ranking de Ticket Médio por Estado

```

SELECT C.ESTADO_UF, CAST(SUM(PG.VALOR_PAGO) /
COUNT(PD.ID_PEDIDO) AS DECIMAL(10,2)) AS TICKET_MEDIO
FROM CLIENTE C INNER JOIN PEDIDO PD ON C.ID_CLIENTE = PD.ID_CLIENTE
INNER JOIN PAGAMENTO PG ON PD.ID_PEDIDO = PG.ID_PEDIDO GROUP BY
C.ESTADO_UF ORDER BY TICKET_MEDIO DESC;

```

-- 22. Crescimento Financeiro: Mês a Mês (Window Function: LAG)

```

WITH FaturamentoMensal AS (
    SELECT YEAR(DATA_PAGAMENTO) AS ANO, MONTH(DATA_PAGAMENTO)
AS MES, SUM(VALOR_PAGO) AS FATURAMENTO
    FROM PAGAMENTO GROUP BY YEAR(DATA_PAGAMENTO),
MONTH(DATA_PAGAMENTO)

```

```
)
SELECT ANO, MES, FATURAMENTO, FATURAMENTO - LAG(FATURAMENTO)
OVER(ORDER BY ANO, MES) AS CRESCIMENTO_BRUTO
FROM FaturamentoMensal;
```

-- 23. Os 3 Produtos com Maior Impacto Financeiro

```
SELECT TOP 3 P.NOME_PRODUTO, SUM(IP.QUANTIDADE *
IP.PRECO_UNIT_VENDA) AS RECEITA_GERADA
FROM PRODUTO P INNER JOIN ITEM_PEDIDO IP ON P.ID_PRODUTO =
IP.ID_PRODUTO
INNER JOIN PEDIDO PD ON IP.ID_PEDIDO = PD.ID_PEDIDO
WHERE PD.STATUS_PEDIDO IN ('Concluído', 'Enviado') GROUP BY
P.NOME_PRODUTO ORDER BY RECEITA_GERADA DESC;
```

-- 24. A Maior Transação Única do Sistema

```
SELECT TOP 1 C.NOME AS CLIENTE_VIP, PG.VALOR_PAGO AS
MAIOR_COMPRA
FROM PAGAMENTO PG INNER JOIN PEDIDO PD ON PG.ID_PEDIDO =
PD.ID_PEDIDO
INNER JOIN CLIENTE C ON PD.ID_CLIENTE = C.ID_CLIENTE ORDER BY
PG.VALOR_PAGO DESC;
```

-- 25. Tempo Médio entre Pedido e Pagamento

```
SELECT PG.METODO, AVG(DATEDIFF(MINUTE, PD.DATA_PEDIDO,
PG.DATA_PAGAMENTO)) AS MEDIA_MINUTOS_PAGAMENTO
FROM PEDIDO PD INNER JOIN PAGAMENTO PG ON PD.ID_PEDIDO =
PG.ID_PEDIDO GROUP BY PG.METODO;
```

-- 26. Matriz Demográfica de Produtos: O que cada Estado Compra?

```
SELECT C.ESTADO_UF, CAT.NOME_CATEGORIA, SUM(IP.QUANTIDADE) AS
QTD_COMPRADA
FROM CLIENTE C INNER JOIN PEDIDO PD ON C.ID_CLIENTE = PD.ID_CLIENTE
INNER JOIN ITEM_PEDIDO IP ON PD.ID_PEDIDO = IP.ID_PEDIDO
INNER JOIN PRODUTO P ON IP.ID_PRODUTO = P.ID_PRODUTO
```

```
INNER JOIN CATEGORIA CAT ON P.ID_CATEGORIA = CAT.ID_CATEGORIA
GROUP BY C.ESTADO_UF, CAT.NOME_CATEGORIA ORDER BY C.ESTADO_UF,
QTD_COMPRADA DESC;
```

-- 27. Análise de Frequência de Compras Repetidas por Cliente

```
SELECT C.NOME, COUNT(PD.ID_PEDIDO) AS QTD_PEDIDOS_TOTAL
FROM CLIENTE C INNER JOIN PEDIDO PD ON C.ID_CLIENTE = PD.ID_CLIENTE
GROUP BY C.NOME ORDER BY QTD_PEDIDOS_TOTAL DESC;
```

-- 28. O Valor da Oportunidade Perdida (Pedidos Pendentes)

```
SELECT PD.STATUS_PEDIDO, SUM(IP.QUANTIDADE * IP.PRECO_UNIT_VENDA)
AS VALOR_FINANCEIRO_PENDENTE
FROM PEDIDO PD INNER JOIN ITEM_PEDIDO IP ON PD.ID_PEDIDO =
IP.ID_PEDIDO
WHERE PD.STATUS_PEDIDO = 'Pendente' GROUP BY PD.STATUS_PEDIDO;
```

-- 29. Produtos Cadastrados vs Disponíveis

```
SELECT (SELECT COUNT(*) FROM PRODUTO) AS TOTAL_CATALOGO,
      (SELECT COUNT(*) FROM PRODUTO WHERE QTD_ESTOQUE > 0) AS
PRODUTOS_DISPONIVEIS,
      (SELECT COUNT(*) FROM PRODUTO WHERE QTD_ESTOQUE = 0) AS
RUPTURA_DE_ESTOQUE;
```

-- 30. Visão Desnormalizada de Auditoria

```
SELECT TOP 20 PD.ID_PEDIDO, C.NOME AS NOME_CLIENTE,
P.NOME_PRODUTO, IP.QUANTIDADE, PG.METODO AS METODO_PAG,
PD.STATUS_PEDIDO
FROM PEDIDO PD INNER JOIN CLIENTE C ON PD.ID_CLIENTE = C.ID_CLIENTE
INNER JOIN ITEM_PEDIDO IP ON PD.ID_PEDIDO = IP.ID_PEDIDO
INNER JOIN PRODUTO P ON IP.ID_PRODUTO = P.ID_PRODUTO
LEFT JOIN PAGAMENTO PG ON PD.ID_PEDIDO = PG.ID_PEDIDO ORDER BY
PD.DATA_PEDIDO DESC;
```

5 CONCLUSÃO

O desenvolvimento deste projeto consolidou a aplicação prática e avançada dos conceitos de bancos de dados relacionais, superando os desafios de estruturação e manipulação de dados em um cenário transacional de alto volume. A transição de um modelo de dados estático para a arquitetura de um e-commerce dinâmico exigiu rigor na normalização (garantindo a integridade referencial até a Terceira Forma Normal) e na definição de chaves e restrições.

A implementação física via Microsoft SQL Server provou que uma modelagem bem executada atua como uma verdadeira "fonte única de verdade", impedindo anomalias de inserção, como demonstrado pelo bloqueio de registros órfãos ou nulos pelas regras de *Foreign Key*. Além disso, a utilização de rotinas T-SQL para a geração orgânica de milhares de pedidos simulou o comportamento real de um ambiente de produção. As consultas analíticas elaboradas — utilizando Common Table Expressions (CTEs), Window Functions e agregações complexas — evidenciam que o banco de dados construído transcende o mero armazenamento, atuando como uma ferramenta robusta para extração de Business Intelligence, capaz de responder a demandas críticas sobre faturamento, retenção de clientes e gestão de estoque.

Sugestões para Possíveis Melhorias:

Visando a evolução contínua da arquitetura desenvolvida, propõem-se as seguintes melhorias para ciclos futuros:

- **Implementação de Triggers (Gatilhos):** Criação de rotinas automatizadas para o abatimento imediato do estoque na tabela PRODUTO assim que um ITEM_PEDIDO for inserido, além de tabelas de log para auditar alterações de preços
- **Encapsulamento em Stored Procedures:** Migrar a lógica de inserção de pedidos para procedimentos armazenados com controle transacional explícito (BEGIN TRAN, COMMIT, ROLLBACK), garantindo que falhas parciais não afetem a consistência do carrinho de compras.

- **Criação de Views Gerenciais:** Abstrair as consultas financeiras mais complexas em exibições virtuais, facilitando a integração futura com ferramentas de visualização de dados como o Power BI.
- **Otimização por Índices:** Implementar índices não-clusterizados (Non-Clustered Indexes) em colunas de alta filtragem, como STATUS_PEDIDO e DATA_PAGAMENTO, para garantir a performance do banco de dados frente a um crescimento exponencial de registros.

REFERÊNCIAS

BEAULIEU, Alan. Aprendendo SQL. 2. ed. São Paulo: Novatec, 2010.

CELKO, Joe. SQL for Smarties: Advanced SQL Programming. 5. ed. Burlington: Morgan Kaufmann, 2014.

DATE, C. J. Introdução a Sistemas de Bancos de Dados. 8. ed. Rio de Janeiro: Elsevier, 2004.

ELMASRI, Ramez; NAVATHE, Shamkant B. Sistemas de Banco de Dados. 6. ed. São Paulo: Pearson Addison-Wesley, 2011.

HEUSER, Carlos Alberto. Projeto de Banco de Dados. 6. ed. Porto Alegre: Bookman, 2009.

MICROSOFT. SQL Server Technical Documentation. Disponível em: <<https://learn.microsoft.com/pt-br/sql/sql-server/>>. Acesso em: 30 maio 2026.